j4_display_right

Pot-label display board for the Johnny 4 robot controller -- the landscape display on the RIGHT of the panel. It sits directly above four potentiometers and acts as an electronic label for them: the bottom of the screen reads **IRIS, COLOR, BRIGHTNESS, VOLUME** (left to right), each with a live value bar. The upper area shows the eight face pots read on the controller (Eyebrow L/R, Basket Eyebrow L/R, Nose, Nose Basket, Bottom Eyelid L/R), each with its 0-255 value and a bar, and the header strip shows the active face (**Face: A**, or **Face: A*** once something has changed since it was saved) and face-preset messages such as **FACE SAVED TO A**.

Since the controller's four ADS1115 ADCs have their channels allocated to the joysticks and face pots (with only spares left for the future 17th-19th pot readouts), this board carries the system's fifth ADS1115, which reads the four pots beneath it and immediately streams the values to the TTGO j4_controller over UART.

What this board does

- Reads 4 pots (IRIS, COLOR, BRIGHTNESS, VOLUME) via a dedicated ADS1115 on its own I2C bus
- Streams the raw readings to the TTGO controller at 25 Hz as an ASCII line: `P:<iris>,<color>,<brightness>,<volume>\n` (the steady stream doubles as this board's heartbeat)
- Renders the face pot values, the label strip and live value bars on a 3.5" ILI9488 TFT in landscape
- Holds the robot's saved faces in `FACES.TXT` on the TFT module's microSD

Where the pot values go from there:

Pot	Path	Effect
IRIS	controller -> receiver -> PCA9685	270-deg iris servo
COLOR	controller -> receiver -> WS2812B strip	LED color: red -> orange -> yellow -> green -> blue -> violet -> white
BRIGHTNESS	controller -> receiver -> WS2812B strip	LED brightness, off to maximum
VOLUME	controller -> receiver -> j4_talk (Teensy)	audio output volume

Hardware

Component	Details
Microcontroller	SeeedStudio XIAO ESP32S3
Display	3.5" TFT LCD, 480x320, SPI, ILI9488 driver (landscape)
ADC	ADS1115, 16-bit, 4 channels, I2C addr 0x48 (ADDR pin to GND)
Storage	microSD in the TFT module's slot, sharing the display's SPI bus. Holds `FACES.TXT`, the authoritative saved-face store for the robot. Format it **FAT32** (exFAT will not mount)

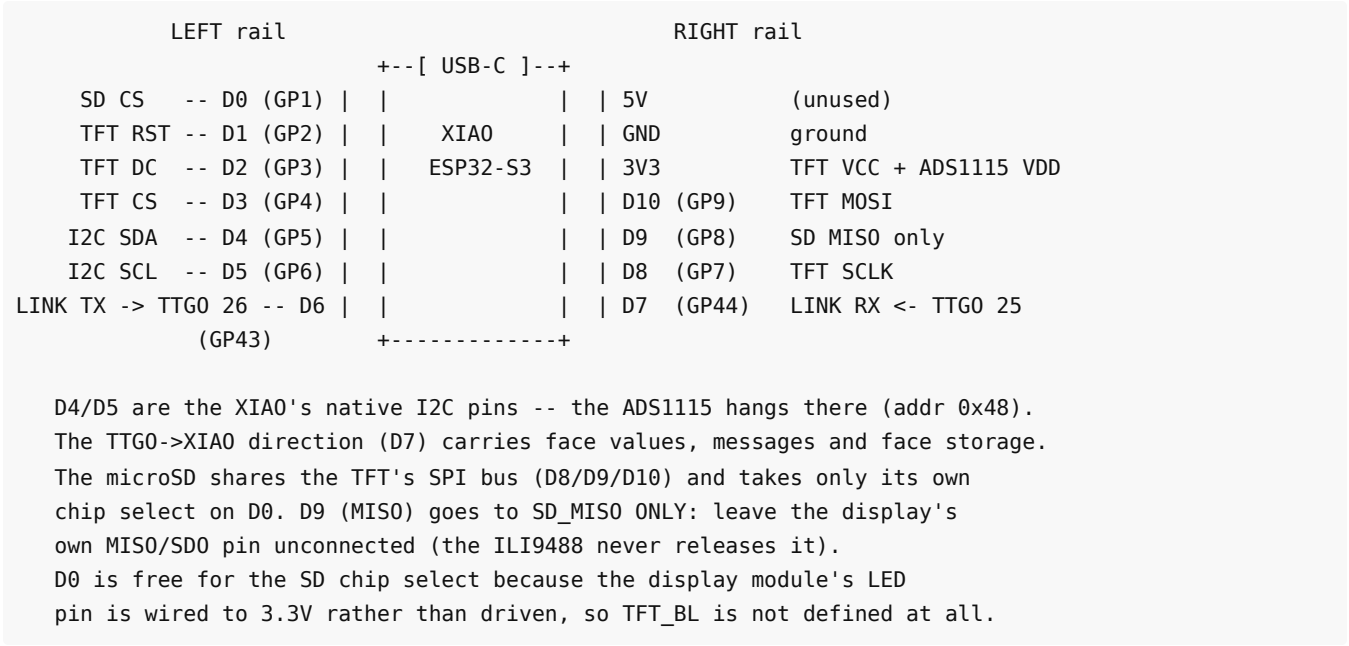
Pin assignments

XIAO Pin	GPIO	Function

3V3	n/a	Display VCC + ADS1115 VDD + microSD VCC (3.3V power)
GND	n/a	Display/ADS1115 GND, shared with TTGO GND
D0	GPIO1	microSD chip select (card on the TFT module's own slot)
D1	GPIO2	TFT RST
D2	GPIO3	TFT DC
D3	GPIO4	TFT CS
D4	GPIO5	I2C SDA (ADS1115)
D5	GPIO6	I2C SCL (ADS1115)
D6	GPIO43	UART TX to TTGO GPIO26
D7	GPIO44	UART RX from TTGO GPIO25 (face values, messages, face store)
D8	GPIO7	TFT SCLK
D9	GPIO8	microSD MISO (SD_MISO only; display MISO/SDO left unconnected)
D10	GPIO9	TFT MOSI

Pin diagram

Physical layout, component side up, USB-C at the TOP. The two header rails read top-to-bottom as shown.



ADS1115 pin diagrams

The four pots under the screen, left to right as labeled on the display: IRIS, COLOR, BRIGHTNESS, VOLUME. Every pot's wiper goes to its A-pin; the outer legs go to 3.3V and GND (all four share the rails).

```
ADS1115 @ 0x48 (ADDR -> GND)               on the XIAO's own I2C bus
+------+
| VDD  |  XIAO 3V3
| GND  |  XIAO GND
| SCL  |  XIAO D5 (GPIO 6)
| SDA  |  XIAO D4 (GPIO 5)
| ADDR |  GND  = address 0x48
| ALRT |  not connected
| A0   |  IRIS pot wiper        -> 270-deg iris servo (PCA9685 on j4_receiver)
| A1   |  COLOR pot wiper       -> WS2812B strip color (j4_receiver)
| A2   |  BRIGHTNESS pot wiper  -> WS2812B strip brightness (j4_receiver)
| A3   |  VOLUME pot wiper      -> j4_talk audio volume
+------+
```

Raw counts stream to the TTGO as `P:<iris>,<color>,<brightness>,<volume>` lines at 25 Hz; the controller scales them with its standard `processPot()` . The IRIS pot shares its function with fader_right on the controller (last-mover-wins, see j4_controller).

Wiring to TTGO controller

3.3V logic on both sides -- no level shifter needed.

XIAO	TTGO
D6 (GPIO43) TX	GPIO26 RX
D7 (GPIO44) RX	GPIO25 TX (face values, messages, face store)
GND	GND

Wiring to ILI9488 display module

XIAO	Display pin
3V3	VCC
GND	GND
(not wired)	LED / BL -- strapped to 3.3V, backlight always on
D1 (GPIO2)	RST
D2 (GPIO3)	DC
D3 (GPIO4)	CS
D8 (GPIO7)	SCK
not connected	MISO (SDO). Leave this pin unwired, see below
D10 (GPIO9)	MOSI (SDI)

The microSD on the same module shares that SPI bus and adds one wire:

XIAO	microSD pin
D0 (GPIO1)	SD_CS
D8 (GPIO7)	SD_SCK (same net as TFT SCK)
D9 (GPIO8)	SD_MISO (the only thing on D9)
D10 (GPIO9)	SD_MOSI (same net as TFT MOSI)
3V3	VCC
GND	GND

If the fitted module does not break out its SD pins, any SPI microSD breakout wires to the same six points with no firmware change.

Do not connect the display's MISO / SDO pin. The ILI9488 never releases that line: its SDO keeps driving it even when the display's CS is high, so with it wired to D9 it fights the card's MISO and the card can never answer. TFT_eSPI's own setup template says so outright: "*Do not connect ILI9488 display SDO to MISO if other devices share the SPI bus (TFT SDO does NOT tristate when CS is high)*". This firmware never reads from the panel, so nothing is lost by leaving it off. (Only the `diag-*` bring-up builds read the panel's ID, and they report all zeros without it, which is harmless.)

The symptom with it connected: a known-good FAT32 card never mounts, power-on shows **NO SD CARD FOUND** in the header, USB serial prints `microSD: no answer`, and every face save ends **SD CARD NOT FOUND, CHECK WIRING**. With it disconnected, power-on shows **SD CARD OK** with the card type and size.

If the card still is not found with the display's MISO off, check that the module's separate **SD header** (SD_CS, SD_MOSI, SD_MISO, SD_SCK, usually at the other end of the board from the display pins) is actually wired: the slot is not connected to the display's SPI pins on the board, so wiring only the display header leaves the card dead.

Card supply on a 3.3V feed (J1). Most of these red 3.5" SPI modules put a 3.3V regulator between the VCC pin and the board, with a solder jumper marked **J1** on the back to bypass it. Fed 5V, the regulator makes 3.3V and J1 stays open. Fed 3.3V (as here), the regulator drops the rail to roughly 2.5-3.0V: the panel still lights and draws normally, but the microSD needs 2.7-3.6V and may never answer. If the module has J1, short it (a blob of solder across the pads) **only while VCC is fed 3.3V**; never feed 5V with J1 shorted. To check without soldering, measure between the card slot's VCC and GND (or the regulator's output pin) with a meter: about 3.3V is fine, under 3.0V is this problem. See "microSD diagnostic" below to confirm.

Wiring to ADS1115

ADS1115	Connect to
VDD	XIAO 3V3
GND	XIAO GND
SCL	XIAO D5 (GPIO6)
SDA	XIAO D4 (GPIO5)
ADDR	GND (I2C address 0x48)
A0	IRIS pot wiper
A1	COLOR pot wiper

A2	BRIGHTNESS pot wiper
A3	VOLUME pot wiper

Each pot's outer legs go to 3.3V and GND (all four share the rails). The gain, data rate, and mode match the controller's own ADS1115s, so the raw counts scale identically (0 to ~17000 full-travel) and the controller can run its standard `processPot()` on them.

Saved-face store

This board carries the authoritative saved-face store for the robot, in `FACES.TXT` on the microSD in the TFT module's slot. j4_controller owns the faces at runtime and keeps a copy in its own NVS flash, but the card here is the source of truth: the controller asks for a dump at boot and writes through on every save.

Protocol

Added to the existing Serial link, alongside `V:` / `M:` / `X:`. Controller to this board:

```
FACES?                                  send me the whole store
FACESAVE:<key>,<v0>..<v10>,<toggles>    persist this face
```

This board back to the controller:

```
FACE:<key>,<v0>..<v10>,<toggles>        one face from the dump
FACE_END:<n>                            dump complete, n faces
FACE_END:-1                             no card present
FACEOK:<key>                            written to the card
FACEERR:<key>                           write failed
```

`FACE_END:-1` is deliberately distinct from `FACE_END:0`. Zero means a mounted card with no faces on it yet, which is a complete and valid answer; minus one means there is no card to read, so the controller stays unsynced and keeps asking. That way a card inserted later is picked up without a reboot.

On-card format is one CSV line per face, keyed by keypad character, which is exactly what j4_talk's Teensy used. A card can be moved between the two stores unconverted.

A save rewrites the whole file rather than appending, so re-saving an existing key replaces that record instead of leaving two for the same key.

Sharing the SPI bus

The card sits on the display's SPI bus, each device with its own chip select. Two details make that safe:

- `SUPPORT_TRANSACTIONS` is mandatory and auto-defined for ESP32 (`Processors/TFT_eSPI_ESP32.h:37`), so TFT_eSPI toggles the HAL mutex around its own transfers.
- The SD library is handed **TFT_eSPI's own `SPIClass`** via `tft.getSPIinstance()`, not the global `SPI` object. TFT_eSPI constructs its own instance (`Processors/TFT_eSPI_ESP32_S3.c:15`), and two instances driving one host fight over its configuration.

The card is clocked at 4 MHz while the panel runs the same bus at 20 MHz. Each transaction sets its own clock, so this is only a conservative choice for a store that writes about 50 bytes at a time.

Absent or failed card

Mounting is guarded the same way as the ADS1115: probe before use, and only retry an absent card every 5 seconds (`SD_REPROBE_MS`). A failed mount runs the SD library's init retries at 400 kHz and holds the loop for a few hundred ms, so probing every second made the bars stutter with no card in; the controller also slows its dump requests to one per 10s while the answer is "no card". A card that mounts is never probed again. A failed write, or a dump that finds the card gone, unmounts it (`SD.end()` , so the next probe really re-initialises rather than returning the stale mount) and answers `FACEERR:` / `FACE_END:-1` , which the controller treats as "held in NVS, retry when the card is back" rather than data loss.

The link's receive buffer is 4096 bytes (`LINK_RX_BUF`), so nothing from the controller is dropped while a repaint or a mount attempt holds the loop.

Background image

The whole screen sits on a 480x320 background image held in flash as RGB565, `src/bg_image.h` . Every draw path blits from it rather than filling a flat colour: the header strip, the labels, the face value numbers, and the cleared portion of each value bar. Text is drawn transparent so the image shows behind it.

Generate the header from a PNG:

```
~/.platformio/penv/bin/python tools/png_to_h.py background.png
```

Anything not already 480x320 is resized. The array costs about 300KB of flash (roughly 10% of the partition) and **no RAM**, since a `const` array on ESP32 stays memory-mapped in flash.

`main.cpp` guards the include with `__has_include("bg_image.h")` . Without the generated header the project still builds and every background blit falls back to a flat `C_BG` fill, so the display behaves exactly as it did before the image existed.

Dithering

`png_to_h.py` applies Floyd Steinberg error diffusion by default. RGB565 carries only 32/64/32 levels per channel, so a smooth gradient collapses into wide flat bands: for `bgscrn_w3.png` the source's 7028 colours became 281, and along a horizontal line through a shaded area the value only changed every 16 pixels. Those steps are what read as contour banding on the panel.

Dithering spends the quantisation error on neighbouring pixels rather than discarding it, so the same 5-6-5 palette resolves into fine noise the eye integrates into a continuous ramp. Measured on that image, transitions along the sampled row went from 30 to 435 and mean absolute error against the source dropped from 2.31 to 1.95.

It is free: same array size, same flash cost, same push time, no firmware change. Pass `--no-dither` to turn it off.

Colour depth ceiling

The ILI9488 accepts 18-bit pixels over SPI, 64 levels per channel, but TFT_eSPI's pixel path is 16-bit RGB565 and expands each value on the way out. Red and blue therefore only ever reach 32 of the panel's 64 levels; green already uses all 64. Recovering the missing bit means bypassing `pushPixels()` and writing raw 3-byte pixels after `setAddrWindow()` , at 50% more flash and 50% longer pushes. Dithered 5-6-5 is smoother than undithered 6-6-6, so this is only worth doing if banding survives dithering.

Byte order

The converter emits **byte-swapped** RGB565 words, and every blit pushes with `swapBytes(false)` . Two independent reasons make that the right combination, and they compound rather than cancel:

1. A `TFT_eSprite` stores colour byte-swapped internally. `TFT_eSprite::drawPixel()` writes `(color >> 8) | (color << 8)`, and `pushSprite()` then sets `swapBytes(false)` *because* the buffer is already swapped, not because the data should be plain.
2. The ILI9488 is a 3-byte RGB panel, so `pushPixels()` with `swapBytes(false)` writes through `tft_Write_16S()`, which swaps its input again on the way out.

Getting this backwards does not produce a subtle tint. Smooth gradients shatter into vivid blue, green and magenta banding while the geometry stays perfect, because the high and low byte of every pixel trade places. If that ever reappears, regenerate the header rather than changing `setSwapBytes()` in `main.cpp`, which is set globally for other reasons and is saved and restored around each blit anyway.

The conversion can be checked without hardware by decoding the generated array through the same byte path the firmware uses and diffing against the source PNG. Correct output gives a max per-channel error of 5, which is RGB565 quantisation. The unswapped version gives 168.

Interaction with the bar band

`drawBar()` needs the background as the base its green fill sits on, so each band sprite is seeded with the matching 480x20 slice of the image at startup and the cleared part of each bar is re-blitted from the image rather than filled black. The delta-push optimisation is unaffected: only the columns between the old and new fill edge still reach the panel.

UART protocol

One ASCII line every 40 ms (25 Hz):

```
P:<iris>,<color>,<brightness>,<volume>\n
```

Values are raw ADS1115 counts. The controller scales them (`processPot()`) and forwards: iris/color/brightness/volume ride the ESP-NOW control packet to j4_receiver, which drives the iris servo and the WS2812B strip and relays volume to j4_talk. Any received `P:` line also marks this board CONNECTED on the controller's connection-status screen.

Downstream (TTGO to this board):

```
V:<11 values>,<key>,<mask>\n  the effective face + active face, 25 Hz (see below)
M:<line1>|<line2>\n    show a message in the header strip ("FACE SAVED TO A" | "SD CARD OK")
X:\n                   clear it
I:<0-255>\n            iris only; sent by controllers before v0_6_41, still accepted
```

`V:` carries the controller's effective face in `FACES.TXT` order: iris, color, brightness, Eyebrow L, Eyebrow R, Basket Eyebrow L, Basket Eyebrow R, Nose, Nose Basket, Bottom Eyelid L, Bottom Eyelid R. Each is 0-255, or `-1` when the controller has no source for that channel (its ADS1115 is absent and no recalled face is holding it), which draws as `--`. The values are post-arbitration and post-recall, so the screen shows what the servos are actually being sent.

The last two fields (j4_controller v0_6_42 and later) are the **active face**: `<key>` is the keypad key last recalled or saved (`-` for none), and `<mask>` says which values differ from what is saved on that key, bit *n* for value *n* in the order above and bit 11 for the LASER / VENT / EYE POP toggles. The header shows **Face: A** while the mask is 0 and **Face: A*** otherwise, each changed face pot number gets a `*` (`123*`), and so do the IRIS / COLOR / BRIGHTNESS labels. A line with only the 11 values (older controllers) is still accepted and simply shows no active face.

- **The eight face pots** (upper area) are read on the controller, so `V:` is their only route here. If it goes quiet for a second they blank to `--` rather than freezing on numbers that look live.
- **IRIS, COLOR and BRIGHTNESS** (bottom bars) are this board's own pots, but the controller is where the IRIS pot is arbitrated against `fader_right` (`dualPick()`) and where a recalled face overrides all three. So while `V:` is live each of those bars draws the controller's value. Once it goes quiet a bar **holds** that value rather than

reverting, and switches to the local pot only when the pot is physically moved past `LINK_TAKEOVER_RAW` (270 counts, the controller's `DUAL_CLAIM_COUNTS` of 4/255 expressed in raw counts). The baseline freezes when the feed stops, so a slow turn still claims the bar, matching `dualPick()`'s frozen-baseline takeover. A `-1` for one of them hands that bar straight back to the local pot.
- **VOLUME** is not part of a face and always draws the local pot.

The upstream `P:` feed still carries this board's own raw counts unchanged. That is what the controller arbitrates against, so it has to keep flowing regardless of what the bars are drawing.

Anything else is treated as line noise from a floating RX pin and ignored; the buffer is length-capped, and a message self-clears after 15 seconds in case the `X:` gets lost. Message timing is owned by the controller, so this board stays dumb and never blocks on the link.

Display layout (480 x 320 landscape)

```
+-------------------------------------------------------------+  y=0
|     KEVCO / Face: A / Face: A* / a message (48px)           |  header strip
+-------------------------------------------------------------+  green line
|   EYEBROW L  |   EYEBROW R  | BASKET BROW L | BASKET BROW R |
|      123     |      123     |      123      |      123      |  face row 0 (92px)
|     [bar]    |     [bar]    |     [bar]     |     [bar]     |
|     NOSE     |  NOSE BASKET | BTM EYELID L  | BTM EYELID R  |
|      123     |      123     |      123      |      123      |  face row 1 (92px)
|     [bar]    |     [bar]    |     [bar]     |     [bar]     |
+-------------------------------------------------------------+  dim line
|     [bar]          [bar]          [bar]           [bar]     |  value bars (36px)
|     IRIS           COLOR        BRIGHTNESS        VOLUME    |  labels (44px)
+-------------------------------------------------------------+  y=320
```

The header strip shows **KEVCO** until a face is recalled or saved, then **Face: A** (with a `*` once anything has changed since that face was saved). It drops back to KEVCO if the controller goes quiet for a second.

Messages from the controller take the header strip's place while they are up: a large first line ("FACE A SAVED TO SD CARD", "OVERRIDE FACE A?", "NO FACE ON A") and a smaller second line ("SD CARD OK", "PRESS * TO CONFIRM, OTHER KEY CANCELS"). Nothing else on the screen is covered. The header returns to its normal text when the controller clears the message, or after 15s if that clear is lost; the controller re-sends an open override prompt every 5s to keep it up, and the header is only repainted when its text actually changes, so the re-send does not flicker.

The face cells run in the order of `FACE_CELLS[]` in `main.cpp`; reorder that table if the physical pots sit differently.

Tuning the layout

Every position, size, font and colour on the screen is a named value in the **SCREEN TUNING** block near the top of `src/main.cpp`. Edit a number there, rebuild, flash, look; nothing else in the file needs touching. The block is grouped by area:

Group	Prefix	What it sets
Colours	`C_`	The palette the items below pick from
Whole screen	`SCREEN_`	Rotation (3, or 1 if the picture is upside down)

Header strip	`HDR_`	Strip height, font, colour and Y of KEVCO / "Face: A" / both message lines
Face grid	`FACE_`	Top of the grid, row height, and for the label, number and bar: font, colour, X nudge, Y offset in the row, bar width and height, the number's box size, divider insets
Bottom row	`BOT_`	Line above the bars, bar size and Y, label font, colour and Y, label dividers
Bars	`BAR_`	Outline and fill colours, gap between outline and fill

Conventions: units are pixels, (0, 0) is the top-left, y grows down. `_Y` values are absolute rows; `_DX` / `_DY` are nudges (+DX right, +DY down). Text Y is the centre of the text. Each item sits centred on its quarter-screen column, so one `_DX` moves that item in all four columns. Face bars and bottom bars are sized independently.

Things to know:

- **Don't let two items overlap.** Each item is erased by repainting the background under its own box, so an overlapping neighbour gets part of itself wiped until the next full repaint.
- **Bigger face numbers need a bigger box.** `FACE_NUM_BOX_W` / `FACE_NUM_BOX_H` is the area each number is erased and drawn in; text larger than it is clipped.
- **Fonts 6, 7 and 8 are digits only**, so the "changed" `*` will not show in them.
- **Values pushed off the screen clip at the edge**; they cannot crash the board.
- **Very tall bars may not fit in RAM.** Each bar row is a full-width sprite (960 bytes per pixel of bar height). If one cannot be allocated, USB serial prints `bar band N sprite FAILED, drawing direct` and that row draws straight to the panel, which works but tears slightly when a pot is swept.

Each row of bars is rendered through its own `TFT_eSprite` canvas, 480 wide by the bar height at 16bpp (19.2KB each at the default 20px, three in all: the bottom row and the two face rows). The face numbers are composed in one small sprite the size of `FACE_NUM_BOX_W` x `FACE_NUM_BOX_H` over a copy of the background and pushed in one go. Nothing is ever composed on the glass, and nothing is redrawn unless its value changed.

Sizing that canvas to the band rather than the whole screen is deliberate, and so is pushing only part of it. The ILI9488 has no 16-bit SPI mode, so TFT_eSPI converts every pixel to 18-bit and sends it through a per-pixel CPU loop (`pushPixels()` in `Processors/TFT_eSPI_ESP32_S3.c`), and the library's own README lists **ILI9488 (DMA not supported with SPI)**. Push cost is therefore strictly proportional to pixels sent, with no DMA to hide it, and the window in which the panel can be caught mid-update *is* the push duration. Pushing a whole-screen sprite every frame would lengthen that window, not shorten it.

So each frame composes into the canvas and then pushes only the columns between the old and new fill edge. A pot sweep moves that edge a few pixels per frame, making a typical update a sliver of roughly 15 x 16 instead of the full 84 x 20 bar: about a sixth of the bytes, and a tear window that shrinks in proportion.

Pushes are wrapped in `startWrite()` / `endWrite()` . The windowed `pushSprite()` renders line by line, and unlike its 1bpp and 4bpp branches its 16bpp branch does not bracket that loop itself, so without the wrapper each row would become its own SPI transaction with its own window command.

If `createSprite()` ever fails, `drawBar()` falls back to drawing direct, which tears but keeps the display usable. Setup prints the sprite status over USB CDC.

Each cell is a quarter of the screen width; the bottom row is centered above its pot. If the image is upside down relative to the panel, change `SCREEN_ROTATION` from 3 to 1 in the SCREEN TUNING block.

Dependencies

Managed via PlatformIO (`platformio.ini`):

Library	Version
Bodmer/TFT_eSPI	latest from GitHub
RobTillaart/ADS1X15	0.3.13

TFT_eSPI is configured entirely through `build_flags` in `platformio.ini` -- no `User_Setup.h` file needed.

Building and uploading

```
# Build
pio run

# Upload
pio run --target upload

# Monitor serial output (USB CDC debug)
pio device monitor
```

If the upload fails, hold the **BOOT** button on the XIAO, tap **RESET**, then release BOOT to force download mode.

TFT bring-up diagnostic

For when the panel is blank while the rest of the board works. TFT_eSPI writes are blind (no ack, and no readback in the init path), so a clean `tft.init()` proves only that the ESP32 shifted the bytes out, not that the panel accepted them. `src/diag/tft_diag.cpp` separates the questions that a blank screen leaves tangled.

It is built by its own environments and excludes `src/main.cpp`, so it touches neither the ADS1115 nor the UART link and runs with the pots and the TTGO unplugged.

```
pio run -e diag-ili9488 --target upload    # then st7796, then ili9486
pio device monitor
```

Environment	Driver under test
`diag-ili9488`	ILI9488 (what the normal firmware assumes)
`diag-st7796`	ST7796
`diag-ili9486`	ILI9486

These red 480x320 modules ship as any of the three with identical silkscreen and identical pinout, and a driver mismatch gives a lit backlight over a blank panel. Flash each in turn: the one that paints identifies the glass.

For the module currently fitted, the manufacturer's listing states ILI9488, `VCC power voltage: 3.3V~5V` and `Logic IO port voltage: 3.3V (TTL)`. So the panel supply is in spec on the XIAO's 3V3 rail, the logic lines connect to the XIAO directly with no shifter, and `diag-st7796` / `diag-ili9486` are insurance rather than the leading theory. There is no level shifter anywhere in the path: not external, and not on the module either, which the manufacturer's own note confirms by telling 5V users to add their own. The logic pins run straight from the XIAO to the ILI9488, which is the best case for signal integrity.

That leaves the clock as a suspect on spec alone rather than on margin: the normal firmware runs `SPI_FREQUENCY=27000000`, above the ILI9488's ~20 MHz write spec, and the diagnostic drops it to 10 MHz to remove the variable.

What it reports:

- **Firmware heartbeat.** The XIAO's on-board user LED (GPIO21, active LOW) blinks at 2 Hz and the USB CDC port narrates every step. Both are independent of the TFT and of the backlight. This matters because if the display's `LED` pin is strapped to 3.3V rather than driven from D0, the backlight is on unconditionally and says nothing about firmware state.
- **Compiled pin map**, so a pasted serial log is self-contained.
- **ID readback** of `RDDID` (0x04) and `RDDST` (0x09) over MISO, which names the controller outright when SDO is wired and driven. Byte 3 of RDDID is 0x88 for ILI9488, 0x86 for ILI9486, 0x96 for ST7796. All 0x00 or all 0xFF means no usable readback, which is common on these modules and not a fault on its own.
- **Solid floods** (red, green, blue, white, black) then a **colour bar and text frame**, cycling continuously. A panel blank through the floods is a different fault from floods that paint in the wrong colours.

Interpreting the result, also printed to serial at the end of each cycle:

Symptom	Meaning
Nothing paints on any of the three drivers	Not a driver mismatch. Check CS/DC/RST/SCLK/MOSI continuity and the module supply
Floods paint	That environment's driver is the correct one
Colours swapped (RED floods blue)	Right driver, wrong colour order. Add `-DTFT_RGB_ORDER=TFT_BGR` or `TFT_RGB`
Inverted or washed out	Try `-DTFT_INVERSION_ON` or `-DTFT_INVERSION_OFF`
Intermittent, tearing, garbage	Clock too high. Raise `SPI_FREQUENCY` back gradually from 10 MHz

Once the panel is identified, carry the working driver and any colour-order or inversion flag into the `[env:xiao-esp32s3]` `build_flags`.

Note on library configuration: TFT_eSPI here is configured entirely by `build_flags`, with no `User_Setup.h`. If those flags failed to reach the library's own translation units the library would silently fall back to its default driver and pins, which produces exactly this blank-panel symptom. They do reach it: the three diag environments build to three different binaries (285440, 287904 and 285360 bytes for ILI9488, ST7796 and ILI9486), which is only possible if `-D<driver>_DRIVER` is changing the compiled library. Worth re-checking with `md5sum .pio/build/diag-*/firmware.bin` after any PlatformIO or TFT_eSPI upgrade.

microSD diagnostic

When power-on keeps showing **NO SD CARD FOUND**, flash the `diag-sd` environment (PlatformIO: Project Tasks > diag-sd > Upload and Monitor). It needs only the XIAO and the TFT module, never draws on the panel (the screen stays blank or white), and repeats three tests every 4 seconds on USB serial:

Line	What it tests	Reading the result
`[1] MISO (D9) idle`	D9 read with the ESP32's pull-up, then pull-down, with both chip selects high	`1, 0` or `1, 1` is fine. `0, 0` means something holds MISO low while deselected: the display's MISO/SDO is still wired, or D9 is shorted to GND

`[2] Raw CMD0 reply`	80 wake-up clocks, then the card's first two commands at 250 kHz, with no library involved	`0x01` = card powered, wired and answering. `0xFF` = no answer at all: an SD_* wire missing or on the wrong pin, no card, or no card power (J1). `0x00` = MISO held low. Anything else = garbled: loose or crossed MOSI/MISO, long jumpers, or low card voltage
`[3] SD.begin`	The SD library's mount at 400 kHz, 1 MHz and 4 MHz (the normal firmware uses 4 MHz), plus a root listing	Mounting only at the slower speeds points at long or loose wires. "card answered, but no FAT32 filesystem" means reformat FAT32 (Windows will not offer FAT32 over 32 GB; use a 32 GB or smaller card or a FAT32 tool)

The loop keeps running, so cards can be swapped and wires fixed while watching the output. Flash `xiao-esp32s3` again afterwards to get the normal firmware back.

Related projects

- **j4_controller** -- the TTGO T-Display this board streams pot data to
- **j4_display_left** -- the portrait jukebox display on the left of the panel
- **j4_receiver** -- where the iris servo and WS2812B strip live